

StudySync AI - Smart Student Productivity Platform

StudySync AI is a gamified productivity application designed to help students track tasks, organize study schedules, generate notes summaries, and take custom practice quizzes.

The platform uses a dark-theme glassmorphism interface running on a Python Flask backend. It integrates Google Gemini 1.5 Flash for notes summarization and quiz generation, and connects to Supabase for authentication and SQL cloud storage.

Key Features

- Smart Task Checklist: Organize assignments and study tasks by course category and priority level. Checking off completed items awards XP and Coins instantly.
 - AI Summary Engine: Input study notes or topic keywords to generate formatted summary sheets, quick revision bulletins, and key takeaways.
 - AI MCQ Quizzer: Generates a 5-question practice quiz based on notes. Includes instant answer verification, score reports, and coins/XP rewards.
 - Quest and Streak Tracker: Tracks consecutive daily login streaks. Includes randomized daily challenges for extra coin rewards.
 - Badge Showcase: Purchase custom profile badges from the shop using earned coins to display them on your academic profile.
 - Live Leaderboard: Displays real-time standings of all registered students sorted by total XP, highlighting your own rank.
 - Study Group Chat: Custom rooms where classmates can join and chat in real-time to collaborate.
 - Synth Sound Effects: Synthesized 8-bit sound effects that trigger on clicks, task completions, quiz answers, chat notifications, and quest unlocks.
-

Architecture and System Updates

Server-Side Chat Synchronization

To address connection drops and rate limits commonly experienced with WebSockets on free-tier hosting (such as Render), the chat system runs on a polling architecture:

- Messages are processed and synchronized through Flask API routes (`/api/chat/messages` and `/api/chat/send`).
- A message ID deduplication set on the client prevents duplicate bubble rendering.
- This ensures real-time delivery across multiple browser windows or devices.

Live Leaderboard System

The leaderboard is populated dynamically from database statistics using the `/api/leaderboard` route:

- Standings are fetched, sorted, and rendered on the client side.
- Highlights your row with special styles and appends a (You) label to help you find your rank instantly.

Web Audio API Synthesizer

Rather than loading heavy audio files (MP3/WAV) that slow down page loads, we use the browser's built-in `AudioContext` to synthesize sound effects dynamically:

- Short frequency burst on clicking navigation links and buttons.
- A rising C5-to-G5 tone on task completion.
- A soft bubble-pop sound on incoming chat messages.
- Clean double-chimes for correct quiz answers and descending sweep for incorrect ones.
- Triumphant fanfare arpeggio on leveling up or claiming rewards.

Cache-Busting Imports

All templates load JS static scripts with a version query parameter (`script.js?v=3`). This forces client browsers to fetch the latest code updates immediately, skipping cached files.

Folder Structure

```
StudySync-AI/
... app.py           # Core Flask server, route controllers, and API routing
... database.py      # Database wrapper (Supabase integration + SQLite local fallback)
... gemini.py        # Gemini API connector (includes structured fallback data)
... requirements.txt  # Backend dependencies
... README.md        # Documentation and setup instructions
... templates/       # HTML templates
.  ... index.html     # Portal landing and user registration
.  ... dashboard.html # Study cockpit, timers, and AI generators
.  ... challenges.html # Leaderboards and peer study chats
.  ... profile.html   # Badges shop and academic dashboard
... static/          # Styles and frontend scripts
... style.css         # Vanilla CSS rules, theme variables, and grid layouts
... auth.js          # User login/signup frontend handlers
... api.js           # Fetch client utility wrapping Flask endpoints
... script.js        # Core DOM controller, charting, and sound synthesizer
```

Local Installation and Setup

1. Clone the repository and navigate into the root directory in your terminal.
2. Create a virtual environment and install dependencies:

```
```powershell
Create environment
python -m venv venv

Activate environment (Windows)
.\venv\Scripts\activate

Install backend packages
pip install -r requirements.txt
```
```

3. Configure environment secrets:

Copy `.env.example` to a new file named `.env` and fill out the values:

```
``bash
FLASK_SECRET_KEY="your-random-cookie-key"
SUPABASE_URL="https://your-supabase-project.supabase.co"
SUPABASE_ANON_KEY="your-supabase-public-anon-key"
GEMINI_API_KEY="your-google-studio-api-key"
``
```

Note: If you leave the credentials empty, the app falls back to Mock Mode. It will initialize a local SQLite file (`studysync.db`) and serve simulated AI summaries/quizzes so you can test the platform instantly.

SQL Database Schema

Run this SQL script in your Supabase SQL Editor or apply it to your database to set up the schema:

```
-- 1. User Profiles
CREATE TABLE public.users (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT UNIQUE NOT NULL,
  full_name TEXT NOT NULL,
  college TEXT,
  course TEXT,
  year_semester TEXT,
  subjects TEXT,
  xp INTEGER DEFAULT 100,
  coins INTEGER DEFAULT 20,
  level INTEGER DEFAULT 1,
  streak INTEGER DEFAULT 1,
  last_active DATE DEFAULT CURRENT_DATE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL,
  badges TEXT DEFAULT 'freshman'
);

-- 2. Tasks Checklist
CREATE TABLE public.tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.users(id) ON DELETE CASCADE NOT NULL,
  title TEXT NOT NULL,
  priority TEXT CHECK (priority IN ('Low', 'Medium', 'High')) DEFAULT 'Medium',
  category TEXT CHECK (category IN ('Study', 'Assignment', 'Revision', 'Other')) DEFAULT 'Study',
  due_date DATE,
  completed BOOLEAN DEFAULT FALSE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

-- 3. Daily Challenges
CREATE TABLE public.challenges (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.users(id) ON DELETE CASCADE NOT NULL,
  title TEXT NOT NULL,
```

```
description TEXT,  
type TEXT CHECK (type IN ('study', 'tasks', 'revision', 'upload', 'quiz')) NOT NULL,  
target INTEGER NOT NULL,  
progress INTEGER DEFAULT 0 NOT NULL,  
completed BOOLEAN DEFAULT FALSE NOT NULL,  
xp_reward INTEGER DEFAULT 50 NOT NULL,  
coins_reward INTEGER DEFAULT 10 NOT NULL,  
date DATE DEFAULT CURRENT_DATE NOT NULL  
);
```

Deployment Guide

Render (Backend Hosting)

1. Push the code to a GitHub repository.
2. Create a new Web Service on Render, and link the repository.
3. Configure these build settings:
 - Environment: Python
 - Build Command: `pip install -r requirements.txt`
 - Start Command: `gunicorn app:app`
4. Go to the Environment tab on Render and paste the environment variables from your `.env` file.